

# **Sistemas Operacionais**

**GERÊNCIA DE PROCESSOS**



# About me...

- Perito Criminal Federal – Polícia Federal
- Doutorando em Informática pela UFPB
- Mestre em Perícias Forenses pela Universidade de Pernambuco
- Especialista em Criminalística Aplicada a Locais de Crime pela Academia Nacional de Polícia.
- Especialista em Docência do Ensino Superior pela Universidade Federal do Rio de Janeiro.
- Graduado em Ciência da Computação pela Associação Paraibana de Ensino Renovado.
- Graduado em Processamento de Dados pela Associação Paraibana de Ensino Renovado.
- Atuou no Serviço Público como Papiloscopista Policial Federal e também como Policial Rodoviário Federal.
- É Oficial da Reserva de 2ª classe do Exército Brasileiro no posto de Primeiro Tenente da Arma de Cavalaria.
- Trabalhou como Analista de Sistemas na multinacional Xerox do Brasil,
- Foi coordenador de informática na TV Cabo Branco (afiliada da Rede Globo).
- Coautor e revisor dos livros Tratado de Computação Forense, Ciências Forenses - Uma Introdução Às Principais Áreas Da Criminalística, Polícia científica transformando vestígios em evidências à luz da cadeia de custódia.
- Aprovações: PRF 2002, PF 2004, MPU 2010, ABIN 2010, PF 2014
- Contato: [gustavopintovilar@gmail.com](mailto:gustavopintovilar@gmail.com)

# Escopo

- Abordar os assuntos mais recorrentes e com fortes tendências para concursos atuais
- Familiarizar o concursando com os tipos de questões mais frequentes.
- Abordar as metodologias de resolução de questões das principais bancas

# **Sistemas Operacionais**

**GERÊNCIA DE PROCESSOS**

# Processos - Conceitos

- Conceito central em qualquer sistema operacional
  - Unidade de gerenciamento de recursos
  - Meio de agrupar recursos relacionados
  - Associado a
    - Espaço de endereçamento (programa + dados + pilha)
    - Conjunto de registradores
  - Há proteção entre os processos: disputa de recursos
  - CPU executa somente um programa por vez
    - Uma CPU = Pseudoparalelismo
    - + de uma CPU/núcleo (multiprocessadores) = Paralelismo

# Processos – Orientações

- É um programa em execução
  - Cada processo tem sua CPU virtual
  - Multiprogramação aumenta o uso efetivo da CPU
  - **CPU Bound**
    - Uma CPU proporcionalmente mais lenta ao número de processos
    - Usa muito processador e pouco IO
    - Executa e vai para fila de pronto
  - **I/O bound**
    - Usa muito IO e pouco processador
    - uma CPU um pouco mais lenta para o processo
    - Melhora o grau de multiprogramação
    - Grau de multiprogramação = quantidade de programas que se revezam em execução
      - Definido pelo escalonador de admissão LP / SCHEDULER
- Multiprogramação depende apenas do sistema operacional
- Multitarefa depende do sistema operacional e do hardware do sistema

# Processos – A vida

- Criação

- Início do sistema
    - Foreground
      - Primeiro Plano
      - Interativo
    - Background
      - Daemons
      - Segundo plano
      - Não interativos, ou seja, o usuário não participa
  - Chamada de sistema de criação de processo por um processo em execução
    - Após a inicialização
    - ajuda no trabalho do processo chamador
  - Requisição do usuário
    - Sistemas interativos
    - Linha de comando ou ambiente iconográfico
  - Início de uma tarefa em lote
    - Somente em computadores de grande porte
    - Execução condicionada à existência de recursos
- OBS: Em todos os casos um processo filho é criado por um processo pai
    - UNIX: Fork
      - processo filho executa chamada execve divide a imagem de memória

- Nenhuma memória para escrita é compartilhada, apenas de leitura
  - Copy-on-write
    - » Imagem de memória iguais até a primeira escrita
    - » Conceito de grupo de processos (Pai, filho, irmãos, netos)
  - Processos pertencem a uma única árvore (init é a raiz)
- Windows: CreateProcess e outras
    - Espaços de endereçamento diferentes desde o início
    - Não existe conceito de hierarquia entre os processos



# Processos – A Morte

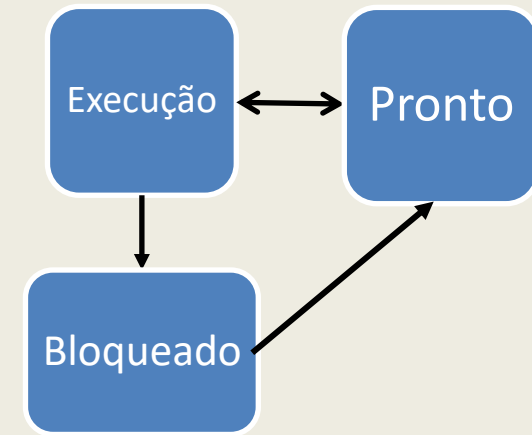


- Saída normal (voluntária)
  - Unix - Exit
  - Windows - ExitProcess
  - Terminou a tarefa
- Saída por erro (voluntária)
  - fornecimento de parâmetros errados
  - Geralmente há uma caixa de diálogo para notificação
- Erro fatal (involuntária)
  - Execução de instrução ilegal
  - referência a memória inexistente
  - divisão por zero
  - Erro pode ser tratado pelo SO ou pelo próprio processo (que sinaliza ao SO sua intenção)
- Cancelamento por outro processo (involuntária)
  - Unix - Kill
  - Windows - TerminateProcess
  - Processo que executa a chamada deve ter autorização necessária



# Processos - Estados

- Execução
  - No processador
  - Não há fila
  - Trap coloca o processo para bloqueado
- Pronto
  - Fila para o processador
  - Revezando a CPU
  - Escalonamento
- Bloqueado
  - Fila para I/O ou E/S
  - Incapaz de executar enquanto evento externo não ocorrer
- OBS: Existem ainda outros estados: Iniciado...terminado (modelo de 5 estados)
  - Modelo estendido - Diagrama de estados



# Implementação do modelo de processos



- SO Mantém uma tabela chamada Tabela de Processos
  - Mudança de estados envolvendo o estado de execução
  - Processo é reiniciado como se nunca tivesse bloqueado.
- Uma entrada para cada processo
- Informações sobre
  - Estado
  - Contador do Programa
  - Ponteiro da Pilha
  - Alocação de memória
  - Estado dos arquivos abertos
  - Enfim: Contexto

# Contextos dos processos



- Hardware
  - registradores gerais
  - registradores PC
    - próxima instrução a ser executada
  - registradores SP
    - Stackpointer
    - Ponteiro para o topo da pilha
  - Registrador psw
    - flags de erros, interrupção, div por 0
- Software
  - Nome do processo
  - PID
  - UID
  - Prioridade de execução
  - Data e hora de criação
  - tempo de processador
  - Quotas
  - Privilégios

# Processos – Espaço usado



- Espaços de endereçamento
  - Range de endereços
  - Lista de posições da memória
  - Varia entre uma quantidade mínima e máxima em que o processo pode ler e escrever
- Dividido em 3 partes
  - Código (ou texto)
    - código do programa
  - Dados
    - variáveis declaradas no programa
    - variáveis não declaradas no programa
  - Pilha
    - Armazena os apontadores do programa



# Threads



- No processo de escalonamento, existem 2 níveis de paralelismo
  - De processos
  - De threads

# Threads de Núcleo

- Núcleo conhece a existência dos Threads e os gerencia
  - Quantum é da Thread
- Desnecessário o uso das tabelas de threads para cada processo e sistema supervisor
- Núcleo os cria ou destrói
- Núcleo possui tabela de processos e de threads
- Mais lentos que os de usuário
- Necessidades de chamadas de sistemas / chaveamento de modos
  - Alternância requer muitos recursos de máquina
  - Custo de chamadas maior
  - Implementado em nível de hardware
  - Escalonador global
- Suporte vem do kernel, usando instruções privilegiadas
- Portabilidade menor, pois é dependente do SO
- Thread bloqueado para E/S não suspende o processo



# Threads de Usuário



- Núcleo só conhece os processos, desconhece as Threads
  - Quantum é do processo
  - Não existe preempção das threads
- Núcleo trata os processos como se fossem monothread
  - Pode ser implementado em SOs que não suportam Threads
  - Cada processo precisa de sua tabela de Threads
- Não há necessidade de chaveamento de contexto, o que facilita seu escalonamento
- Thread precisa liberar voluntariamente a CPU para que outro o possa fazer
  - Thread do mesmo processo deve ceder voluntariamente seu lugar - yield
  - Escalonador nunca fará isso
  - Thread bloqueante paralisa outros threads do processo
- Executadas instruções que não precisam de privilégios
- Menor sobrecarga
  - Alternância de threads requer poucos recursos de máquina
- SOs não escalona as threads, apenas os processos
  - Necessidade do sistema supervisor
  - Aplicação escalona usando bibliotecas adequadas
  - Cada processo pode possuir seu algoritmo de escalonamento personalizado



# Threads – Implementação Híbrida

- Núcleo conhece apenas as Threads de núcleo
- Alguns desses threads multiplexam os threads de usuário
  - Número de Threads de núcleo nunca pode ser maior que o número de threads de usuário
  - Modelo Muitos-Para-Um
    - mapeia muitos threads de nível de usuário para threads do kernel
  - Modelo Um-Para-Um
    - cada thread de usuário para um thread de kernel
    - O modelo de threading suportado por um kernel Linux típico é do tipo um-a-um.
  - Modelo Muitos-Para-Muitos
    - multiplexa muitos threads de nível de usuário para um número menor ou igual de threads de kernel
    - Tenta combinar as vantagens e eliminar as desvantagens



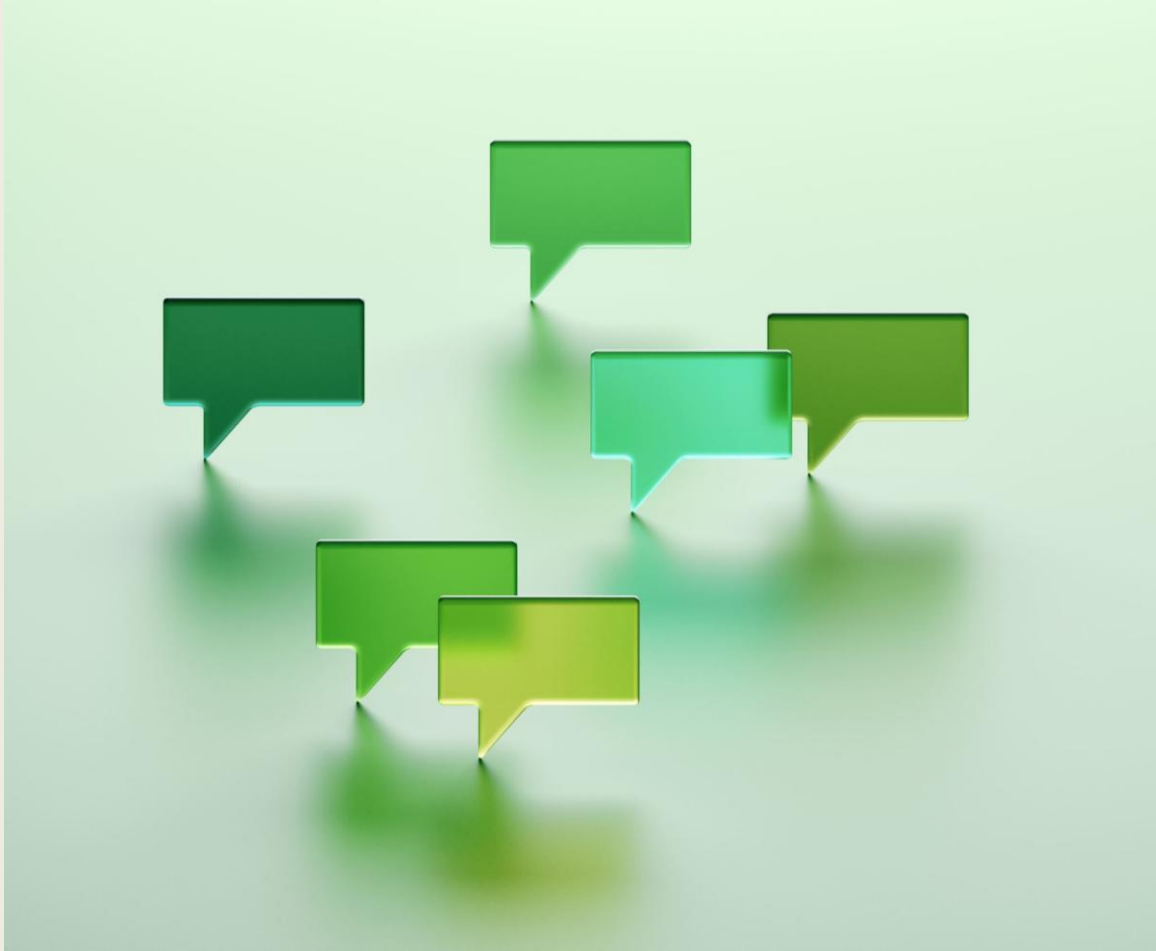


# Threads – Pontos relevantes

- São mais fáceis de criar e destruir que os processos
- Inexistência de recursos associados
- 100 x mais rápido do que criar um processo
- Custo de gerenciamento bem menor que processos (comunicação mais simples)
  - Ganho de desempenho só ocorre com a IO/bound
- Úteis em sistemas multiprocessados, onde o paralelismo real é possível
- Processos são usados para agrupar recursos (unidade de gerenciamento de recursos), Threads para escalonamento de CPU
- Escalonamento similar aos dos processos
- Aplicações típicas
  - Servidor web
  - Processador de texto



# Comunicação Interprocessos - IPC



- Processos precisam se comunicar entre si
  - Sinais
  - Mensagens
- Questões
  - Como um processo passa informação para outro?
    - Para os Threads do mesmo processo é fácil, pois compartilham o mesmo espaço de endereçamento
  - Como garantir que 2 ou + processos não entrem em conflito?
  - Sequência adequada quando existirem dependências

# Processos – Condições de corrida

- Processos que trabalham juntos podem compartilhar algum armazenamento comum (ler / escrever)
  - RAM
  - Arquivo
- 2 ou + processos lendo ou escrevendo algum dado compartilhado
- Problema do produtor consumidor
- Um processo só pode usar algo que já foi produzido pelo produtor

# Processos - Concorrência

- Existe quando não há paralelismo
- Pode existir quando há paralelismo
- Problemas
  - Bloqueio de recursos
  - Adiamento indefinido
  - Intermitentes - Quando o paralelismo não previu o bloqueio de recursos
  - Solução - Sincronização entre processos e threads
- Sinais
  - Interrupções de software para notificar ocorrência de eventos
  - Usado para sincronia
  - Não há troca de informações, apenas aviso
  - variável, normalmente binária
    - Mutex - se for binária
    - Semáforo - Variável incrementável
- Mensagens
  - Há troca de informações

# Processos – As inconsistências

- Problema da conta bancária
- Evitadas através de exclusão mútua
  - Garantir o uso da região crítica por um processo
    - Garante a multitarefa
    - Acesso ao mesmo recurso
  - Nada mais é do que uma variável indicando se está na região crítica
    - 0 ou 1 significando down e up / P e V
  - Problemas causados pela Exclusão mútua
    - Velocidade de execução de processos
    - Gargalo onde o mais rápido espera pelo mais lento
    - Starvation
    - Sincronização condicional
- Soluções
  - Semáforos
  - Monitores
  - Trocas de mensagens



# Processos – Região Crítica

- Região **do código** que deve ser executada de forma atômica
- Apenas um processo por vez na seção crítica
- Parte do programa em que há acesso à memória compartilhada é chamada região crítica ou seção crítica
- Impedir que mais de um processo leia e escreva ao mesmo tempo na memória compartilhada - Exclusão Mútua
- Se 2 processos nunca estivessem em suas regiões críticas ao mesmo tempo, as disputas seriam evitadas
- Serialização

# Processos – Região Crítica

- **Solução em hardware**
  - Desabilitar as interrupções
    - Existem operações de interrupções que podem ser mascaráveis
    - Programador deve se preocupar com a habilitação
    - Não é uma boa opção
  - **Uso da instrução do processador (flag) Test-and-Set**
    - Implementada em hardware
    - Espécie de variável global
    - Manuseio via SO

# Processos – Região Crítica

## Solução em Software

– Algoritmo de Lamport

– Semáforos

- Variável inteira não negativa (down, up): Valores 0 ou maiores
- Operações Down (sleep / P) e Up (wakeup / V)
- Se binário = mutex
  - Quando não é preciso usar a capacidade do semáforo de contar
  - Não controla quantos consumidores existem
  - Variável que pode estar em um dos dois estados: Impedido ou Desimpedido

# Processos – Região Crítica

## Solução em Software

### – Monitores

- Mecanismo de sincronização
- Procedimento que troca parâmetros com o processo
- Cada processo em estado de pronto chama o monitor
- Garante a execução da região crítica de cada processo

# Deadlock

- Processos bloqueados ad infinitum
- Ocorrem em recursos de hardware ou software
  - Falha de comunicação entre processos. A falha é do programa e não do S.O.
- "congestionamento de processos"
- condição específica quando dois ou mais processos estão esperando cada um pelo o outro para liberar um recurso
- mais de dois processos estão à espera de recursos em uma cadeia circular
- problema comum em multiprocessamento onde muitos processos compartilham um tipo específico de recurso mutuamente exclusivo
- Solução: hardware (ou hard lock), que garante o acesso exclusivo aos recursos pelos processos, forçando o acesso serializado



# Deadlock

- Não se confunde com adiamento indefinido / starvation / inanição
- Starvation
  - Processo nunca obtém o serviço
  - NÃO está necessariamente em deadlock
  - Preterição indefinida
  - Evitada com FCFS
  - Resolvido através do aging
- A diferença principal é o estado do processo
- No deadlock os processos estão bloqueados, já no starvation estão em execução ou prontos

# Deadlock

- Deadlock <> Livelock
  - Livelock - Ação
    - Programa A, chama programa B que por sua vez chama A, ou processo está em algum loop infinito interno
  - Deadlock - Paralisação
    - Impasse

# Deadlock – O Problema

- Condições OBRIGATÓRIAS (necessárias e suficientes) para criação de um cenário propício ao deadlock.
- 1.Exclusão mútua
  - Ou um recurso está associado a um processo ou está disponível
  - Um recurso que não pode ser usado por mais de um processo de cada vez
- 2.Posse e espera
  - Processos que retêm recursos podem solicitar novos antes da liberação
  - Processos que já possuem recursos podem solicitar novos recursos mantidos por outros processo
- 3.Não preempção
  - Nenhum recurso pode ser retirado à força de um processo, os recursos só podem ser liberados pela ação explícita do processo
  - Recursos concedidos devem ser liberados pelo processo requisitante e não tomados
  - Não haverá interferência do SO
  - Preemptivel: Pode ser retirado do processo, Não há deadlock
  - Não preemptivel: Não pode ser retirado, Há deadlock
- 4.Espera circular
  - dois ou mais processos formar uma cadeia circular onde cada processo espera por um recurso que o próximo processo na cadeia detém

# Deadlock – A solução

- Estratégia preventiva
  - Negar uma das 4 condições
    - Exclusão mútua - Contornado com spooling
    - Posse e espera - Requisitar TODOS recursos com antecedência. Ou tudo ou nada
    - Não preempção - Impossível ou a pior de todas
    - Espera circular - Um recurso por vez

# Deadlock – A solução

- Estratégia Corretiva
  - Detecção
  - Muitas vezes, a prevenção não pode ser usada
  - Detectando um impasse ocorrido é facilmente possível, pois os recursos que cada processo tem bloqueado e / ou solicitadas são atualmente conhecidos
  - Recuperação: Encerramento de um dos processos e consequente rollback
- OBS: Ignorar o problema até que ele se estabeleça (Unix e windows) - devido ao alto custo envolvido na estratégia preventiva

# QUESTÕES

GERÊNCIA DE PROCESSOS



**Ano: 2025 Banca: Avança SP Órgão: Prefeitura de Morungaba - SP Prova: Avança SP - 2025 - Prefeitura de Morungaba - SP - Técnico em Informática**

1. O termo “processo” foi usado inicialmente pelos projetistas do SO MULTICS na década de 1960. Esse é um termo um pouco mais generalizado do que “job”. Muitas definições têm sido dadas para o termo processo, incluindo:

- I) Um programa em execução.
- II) O “espírito animado” de um programa.
- III) A entidade à qual um processador é atribuído.

De acordo com o texto acima, o termo “processo” pode, em outras palavras, ser melhor descrito como:

Alternativas

- A - Um programa em execução que não possui recursos alocados.
- B - A unidade básica de trabalho executada por um sistema operacional, representando um programa com seus recursos associados.
- C - Exclusivamente a parte de código de um programa em execução.
- D - A unidade de troca bit a bit.
- E - A entidade responsável por alocar memória física para todos os programas em execução.

**Ano: 2024 Banca: IF-TO Órgão: IF-TO Prova: IF-TO - 2024 - IF-TO - Analista de Tecnologia da Informação**

2. Em relação a processos é correto afirmar que:

- A - Um programa em disco sempre ocupa mais espaço em uma mídia de armazenamento persistente do que quando em execução.
- B - Um programa ao ser executado gera um processo com nenhuma, uma, ou várias threads.
- C - O comportamento dos sistemas operacionais de hoje (Windows e Linux) é executar processos e não threads.
- D - As threads se comportam como alocador de recursos no sistema operacional e os processos como fluxos de execução do código que fazem uso dos recursos alocados pelas threads.
- E - Um programa ao ser executado pode gerar mais de um processo no sistema operacional.

**Ano: 2024 Banca: IV - UFG Órgão: TJ-GO Prova: IV - UFG - 2024 - TJ-GO - Analista Judiciário - Área Especializada - Analista de Sistemas**

3. Considerando os sistemas operacionais modernos, a situação de Starvation ocorre quando

A - um processo é sempre escalonado para executar, pois ele possui prioridade de tempo real.

B - um processo nunca é executado, porque fica bloqueado esperando a execução de algum outro processo.

C - um processo sempre executa em um determinado núcleo de execução, em virtude da falha de escalonamento no sistema.

D - um processo nunca é executado, pois a existência de processos com maior prioridade o impede de entrar em execução.

**Ano: 2024 Banca: FGV Órgão: Prefeitura de Caraguatatuba - SP Prova: FGV - 2024 - Prefeitura de Caraguatatuba - SP - Técnico em Processamento de Dados**

4. Considerando os problemas de concorrência, assinale a afirmativa correta.

A - Mutual Exclusion é uma das condições para Deadlock.

B - Deadlock é uma situação em que dois ou mais processos são lentificados na progressão devido às suas demandas de recursos mutuamente incompatíveis.

C - Na Mutual Exclusion um processo pode receber acesso preferencial aos recursos.

D - No Deadlock um conjunto de processos não consegue progredir devido às suas demandas de recursos incompatíveis logicamente.

E- Mutual Exclusion implica em um ciclo de recursos e processos em que cada processo está aguardando recursos que são mantidos pelo próximo processo no ciclo.

**Ano: 2024 Banca: CESGRANRIO Órgão: Caixa Prova: CESGRANRIO - 2024 - Caixa - Técnico Bancário  
Novo - Tecnologia da Informação - Rio Grande do Sul**

5. A sincronização é um mecanismo que garante a comunicação entre processos concorrentes e o acesso a recursos compartilhados. Dentre as soluções para sincronização, a exclusão mútua impede que dois ou mais processos acessem o mesmo recurso no mesmo instante.

A implementação da exclusão mútua usa semáforos do tipo mutex, que assumem APENAS

A - dois valores, por exemplo, 0 e 1

B - três valores, por exemplo, 0, 1 e -1

C - quatro valores, por exemplo, 0, 1, -1 e -2

D - cinco valores, por exemplo, 0, 1, -1, -2 e -3

E - seis valores, por exemplo, 0, 1, -1, -2, -3 e -4

**Ano: 2024 Banca: Ibest Órgão: CRQ - 13ª Região - SC Prova: Ibest - 2024 - CRQ - 13ª Região - SC - Assistente Administrativo (Operador de Computador)**

6. No que diz respeito a computadores e Sistemas Operacionais, julgue o item.

A gerência de processos é uma das principais funções de um sistema operacional, que envolve a criação, o escalonamento e a terminação de processos.

Certo

Errado

**Ano: 2024 Banca: CESPE / CEBRASPE Órgão: TCE-AC Prova: CESPE / CEBRASPE - 2024 - TCE-AC -  
Analista de Tecnologia da Informação - Área: Infraestrutura de TI**

7. Em relação à gestão de processos e à administração de sistemas operacionais, julgue o item que se segue.

Em relação aos estados de um processo em execução, o estado bloqueado ocorre quando o administrador do sistema operacional determina que o processo espere a conclusão de um processo prioritário.

Certo

Errado



**Ano: 2024 Banca: CESPE / CEBRASPE Órgão: TCE-AC Prova: CESPE / CEBRASPE - 2024 - TCE-AC -  
Analista de Tecnologia da Informação - Área: Infraestrutura de TI**

8. Em relação à gestão de processos e à administração de sistemas operacionais, julgue o item que se segue.

Como parte integral do funcionamento do sistema operacional, um processo pode ser criado e terminado, mas não pode ser agendado porque, em caso de agendamento, é necessário que a parte do usuário (user space) seja invocada.

Certo

Errado

**Ano: 2024 Banca: FADURPE Órgão: UFRPE Prova: FADURPE - 2024 - UFRPE - Analista de Tecnologia da Informação/Área suporte Redes**

9. No gerenciamento de processos em sistemas operacionais, qual é o estado de um processo que está pronto para ser executado, mas ainda não foi escalonado pela CPU?

A - Executando

B - Bloqueado

C - Suspenso

D - Pronto

E - Terminado

**Ano: 2024 Banca: IF-ES Órgão: IF-ES Prova: IF-ES - 2024 - IF-ES - Professor EBTT - Computação**

10. Acerca dos Sistemas Operacionais, assinale a opção que corresponde à seguinte situação:

Um processo A solicita permissão para usar um recurso X, que lhe é concedido. Um processo B solicita permissão para usar um recurso Y, que também lhe é concedido. E, então, o processo A solicita acesso ao recurso Y, mas a requisição é suspensa até que B o libere. Entretanto, ao invés de liberar o recurso Y, o processo B requisita acesso ao recurso X.

A - Impasse (Deadlock).

B - Região Crítica.

C - Condição de Corrida.

D - Inanição (Starvation).

E - Exclusão Mútua.

# Gabarito



1. B

2. E

3. D

4. A

5. A

6. C

7. E

8. E

9. D

10. A